

開発記録：科学文献からの数式抽出パイプライン

AutoPMob プロジェクト

更新日：2026 年 6 月 2 日

目次

1	概要	3
2	遭遇した問題と解決策	3
2.1	API モデル 404（モデル未提供）	3
2.2	レートリミット（429 RESOURCE_EXHAUSTED）	3
2.3	数式内の変数表記の揺れ（ c_w と c_p ）	4
2.4	変数表記の統一（訓練用正規化）	4
3	技術メモ	4
4	各ファイル・ディレクトリの役割	5
4.1	プロジェクトルート（実行スクリプト・メタファイル）	5
4.2	設定・運用補助	6
4.3	データ・実験成果物（リポジトリに含まれる例）	6
4.4	ドキュメント	7
5	データセット定義と評価（次の開発者向けガイド）	7
5.1	データ構造（Equation と Training Case）	7
5.2	タスク定義（Equation Retrieval / Model Selection）	7
5.3	なぜ MRR を使うか（採用理由）	7
5.4	MRR と Recall@K の計算方法	7
5.5	データ分割（リーク防止のための paper/source_id split）	8
5.6	再現可能な実験手順（何をどう動かすか）	8
5.7	現時点の結論（次に何をすべきか）	8
6	GNN の仕組みと埋め込み（次の開発者向け）	8
6.1	なぜ現状のグラフ構築では GNN の強みを活かさないか（分析）	8
6.2	グラフの構築（build_graph_data.py → equation_graph.pt）【二部グラフ版】	9
6.3	式の埋め込みの 2 系統	9
6.4	GNN の役割（2 種類の使い方）	10
6.5	スコアリングと学習	10
6.6	ファイルと役割の対応	10
7	今後の予定・TODO	10

7.1	試したいこと・実験アイデア（大胆に）	11
8	更新履歴	11

1 概要

本ドキュメントは、GNN（グラフニューラルネットワーク）を用いた物理モデル自動構築の研究の一環として開発している、科学文献 PDF から数式と変数定義を抽出するパイプラインの開発記録である。記録内容は研究室ゼミの報告および将来の研究論文の素材として利用する。

パイプラインの目的

- 科学文献の PDF を入力とし、番号付き数式および付録の数式を LaTeX 形式で抽出する。
- 各数式について変数定義（variable definitions）と物理的意味（context）を周辺テキストから抽出する。
- 抽出結果を JSON 形式で保存し、物理モデル自動生成モデルの訓練データとして利用する。

技術スタック

- API: Google Gemini API (`gemini-2.5-flash`)
- SDK: `google-genai` (PDF 直接アップロード、Structured Output 対応)
- スキーマ検証: `Pydantic`
- 出力: `extracted_equations.json` (および表記正規化版 `extracted_equations_normalized.json`)

2 遭遇した問題と解決策

2.1 API モデル 404（モデル未提供）

現象

- `gemini-1.5-pro` を指定して `generateContent` を呼ぶと、API から 404 NOT_FOUND が返る。
- エラーメッセージ: “models/gemini-1.5-pro is not found for API version v1beta”。

原因

- 利用環境 (API version v1beta) では `gemini-1.5-pro` が提供されていない、またはモデル ID が変更されている。

対応

- 呼び出しモデルを `gemini-2.0-flash` に変更。のちに無料枠の都合で `gemini-2.5-flash` に変更。
- 利用可能なモデル一覧は <https://ai.google.dev/gemini-api/docs/models> 等で確認する。

2.2 レートリミット (429 RESOURCE_EXHAUSTED)

現象

- リクエスト直後に 429 RESOURCE_EXHAUSTED が返る。
- メッセージに “Quota exceeded for metric: ... generate_content_free_tier ..., limit: 0” とあり、無料枠のリクエスト数・トークン数が 0 に制限されている。

原因

- 無料ティアのクォータが枯渇している、または当該プロジェクトでは `gemini-2.0-flash` の無料枠が 0 に設定されている。

対応

- 利用可能なモデルに変更: `gemini-2.5-flash` では「1 分あたり 5 回 (5 RPM)」の枠が利用可能であったため、モデルを `gemini-2.5-flash` に統一。
- 5 RPM を超えないよう、`generate_content` の呼び出し前に「前回呼び出しから最低 12 秒 (60/5) 経過するまで待機」する処理を追加。
- リトライ回数・待機時間は当初「12 回・最大 300 秒」としていたが、ユーザ体験を考慮して「5 回・初

回 15 秒・最大 60 秒」に短縮（レートリミット時のみリトライ）。

2.3 数式内の変数表記の揺れ (c_w と c_p)

現象

- 論文では 5 番・6 番の式で「水の比熱」を明示するため c_w を用いているが、抽出結果ではすべて c_p になっていた (eq_5, eq_6)。

原因 (推定)

- LLM の記号正規化**：熱力学では比熱の記号として c_p が一般的なため、モデルが c_w を c_p に「統一」して出力した可能性。
- PDF の見た目**：フォントによっては下付きの w と p が似て見え、誤読した可能性。
- 意味の同一視**：「水で満たされている」という文脈で c_w と c_p を同一視し、 c_p に揃えた可能性。

対応

- プロンプトを強化：「変数記号・添字は文献の表記をそのまま抽出する。 c_w を c_p に変更しない」旨を明記。
- 訓練用には「同じ物理意味の変数は正規化する」方針とし、`config/notation_map.json` と `normalize_notation.py` を用意。 $c_w/C_p/c_p$ 等を c_p に統一する後処理を導入。抽出はできるだけ原文のまま、訓練前に正規化する二段構えにした。

2.4 変数表記の統一 (訓練用正規化)

要望

- 物理モデル自動生成モデルの訓練のため、同じ物理意味を持つ変数の表記をすべて統一したい（モデル誤読・論文側の表記揺れのいずれも吸収）。

対応

- `config/notation_map.json` で「正規表記 (canonical)」と「揺れ (alternatives)」を定義。
- `normalize_notation.py` で、抽出済み JSON の `equation` および `variables` のキー・説明文に置換を適用し、`*_normalized.json` を出力。
- 抽出パイプラインに `-normalize` オプションを追加し、抽出直後に正規化版を保存できるようにした。

3 技術メモ

- Structured Output**: Pydantic の `model_json_schema()` で配列スキーマを組み、`response_mime_type: application/json` と `response_json_schema` を渡して JSON 配列を取得。
- ファイルアップロード**: `client.files.upload()` で PDF をアップロードし、`state` が `ACTIVE` になるまでポーリング。`Part.from_uri()` でコンテンツに含める。
- 環境変数**: `GEMINI_API_KEY` または `GOOGLE_API_KEY`。オプションで `EXTRACT_EQUATIONS_MIN_INTERVAL` (5 RPM 用)、`EXTRACT_EQUATIONS_MAX_RETRIES` 等。
- 開発記録の自動追記と GitHub 同期**: Cursor のルール (`.cursor/rules/development-log.mdc`) で、**全ての變動** (機能追加・新スクリプト・バグ修正・設定変更等) の内容と理由を `development_log.tex` の「更新履歴」に追記するよう指示。あわせて、変更後は必ず `git add`、`git commit`、`git push` を実行し、GitHub へ同期する運用とする。
- コミット時の自動 push**: `.git/hooks/post-commit` で `git commit` のたびに `git push` を実行するため、コミットすればリモートへ反映される。
- 統合パイプライン** `build_equation_graph.py`: `input_pdfs/` 内の「未抽出」PDF のみ一括処理 (処

理済みは `processed_pdfs.json` で管理、1 ファイルごとに 15s 待機で 5RPM 対策)。LLM 知識から化学工学基礎式を 20 個生成 (Handbook)。抽出・生成結果を単一の `unified_equations.json` に統合 (既存時は追記、`source_id+eq_id` で重複排除)。-mode pdf|handbook|all、-processed-list で処理済み一覧を指定可能。

- **API モデル**: 当初 `gemini-1.5-pro`、のち `gemini-2.5-flash` (5 RPM)、RPD 制限対策で `gemini-2.0-flash` に変更後、高性能化のため `gemini-2.5-pro` に統一。
- **デフォルト API キー**: 環境変数未設定時に限り、開発用デフォルトキーを使用するように変更 (`extract_equations.py`、`build_equation_graph.py`、`test_api.py`)。
- **論文メタデータ取得** `fetch_papers.py`: Semantic Scholar API でクエリ A/B 各 50 件のメタデータを取得し、`paper_dataset_links.json` に保存。オープンアクセス PDF URL を `has_open_access_pdf`、`open_access_pdf_url` で明示。429 時はリトライと `SEMANTIC_SCHOLAR_API_KEY` 対応。
- **グラフ前処理** `build_graph_data.py`: `unified_equations.json` から式-変数二部グラフの Data を構築し `equation_graph.pt` に保存 (詳細は各ファイル・ディレクトリの役割)。
- **訓練ケース生成** `generate_training_cases.py`: LLM で約 150 件のコアカスを生成 (`context`、`input_variables`、`output_variables`、`correct_model_ids`)。正解モデルは「入力を与えられれば出力を解ける」「十分かつ最小限の数式群」となるようプロンプトで制約。Python で入出力スワップ・ランダム入出力・context 言い換えにより 1 コアから 5-6 バリエーションを生成し、900 件以上を `training_cases.json` に保存。
- **可視化・分析** `analyze_dataset.py`: 数式グラフ・訓練ケースの統計と可視化。ドメイン分布、次数分布、埋め込みの PCA (ドメイン色分け)、変数共起ネットワーク、訓練ケースの variant 分布・正解モデル数分布。論文付録向け `dataset_report.txt` と `figures/*.png` を出力。

4 各ファイル・ディレクトリの役割

リポジトリ内の主要なパスと、その責務の一覧である (.gitignore 対象や個人環境依存は除く)。

4.1 プロジェクトルート (実行スクリプト・メタファイル)

- `README.md`: 環境構築、代表的なコマンド、リポジトリ構成の入口。
- `CLAUDE.md`: 別ツール・別環境で開発を続けるときの引き継ぎ要約 (目的・データ・実験結論・スクリプト対応)。
- `research_context.md`: 研究メモ用 (`CLAUDE.md` へのポインタを含む)。
- `requirements.txt`: Python 依存パッケージの一覧。
- `extract_equations.py`: 単一 PDF を Gemini API に渡し、数式・変数定義・周辺文脈を JSON で抽出する。
- `normalize_notation.py`: 抽出済み JSON の式文字列・変数キー等を `config/notation_map.json` に従い訓練向けに正規化する。
- `build_equation_graph.py`: `input_pdfs/` の未処理 PDF を一括抽出し、必要に応じ Handbook 式を LLM 生成し、`unified_equations.json` に統合する (スキップ管理は `processed_pdfs.json`)。
- `fetch_papers.py`: Semantic Scholar API で論文メタデータを取得し `paper_dataset_links.json` に保存する。
- `build_graph_data.py`: `unified_equations.json` から式ノード・変数ノードの二部グラフを構築し、各ノードに CodeBERT [CLS] (768 次元、変数側は含有式の平均) を付与して `equation_graph.pt`

(PyTorch Geometric) を出力する。

- `generate_training_cases.py`: LLM でコアケースを生成し、入出力スワップ等でバリエーションを増やして `training_cases.json` を構築する。
- `validate_dataset.py`: 式のグローバル ID 参照、ケースと式の変数整合、重複などを検査し `validate_dataset_report.json` を出力する。
- `add_context_paraphrases.py`: 訓練ケースの文脈のみ言い換えたバリエーションを `training_cases.json` に追加する。
- `analyze_dataset.py`: グラフ・訓練ケースの統計と図表を生成し、`dataset_report.txt` と `figures/` に書き出す。
- `baseline_retrieval_eval.py`: TF-IDF、変数 Jaccard、混合ベースライン等の retrieval 評価ロジック (単体実行および実験ランナーから利用)。
- `run_retrieval_experiments.py`: 複数の scoring 方式を同一の `source_id` 分割で学習・評価し、結果を `experiments/` に JSON と Markdown で保存する。
- `train_gnn_retriever.py`: `equation_graph.pt` 上で式側を GCN または GAT で更新し、クエリを SVD→MLP で揃えて InfoNCE 型で学習する簡易スクリプト (レポートは `gnn_train_eval_report.json`)。
- `train_gnn_refine_svd.py`: 式・クエリを同一 TF-IDF+SVD 空間に置き、式埋め込みを差 GCN で更新する実験用スクリプト (`gnn_refine_svd_report.json`)。
- `test_api.py`: Gemini API の疎通確認用。

4.2 設定・運用補助

- `config/notation_map.json`: 同一物理量の記号揺れを正規表記に寄せるための対応表。
- `scripts/install-git-hooks.sh`: コミット後に `git push` する `post-commit` フックを入れるためのインストール用シェルスクリプト。
- `.cursor/rules/development-log.mdc`: コード変更時に本 `development_log.tex` を更新し GitHub に同期するよう指示するエディタ用ルール。

4.3 データ・実験成果物 (リポジトリに含まれる例)

- `unified_equations.json`: 全ソースを統合した数式ライブラリ (グラフ構築・訓練ケースの参照元)。
- `training_cases.json`: retrieval タスクの入力 (文脈・I/O 変数) と正解 ID 集合。
- `equation_graph.pt`: PyG Data (x, 二部 `edge_index`, `num_equations`, `edge_index_eq_eq` 等)。
- `processed_pdfs.json`: `build_equation_graph.py` が既に処理した PDF ファイル名のリスト。
- `paper_dataset_links.json`: `fetch_papers.py` の取得結果 (PDF URL 等)。
- `extracted_equations.json`: 単一 PDF 抽出の典型出力 (中間・サンプル用)。
- `experiments/retrieval_experiments.json`: 方式比較実験の分割・ハイパラ・指標の機械可読ログ。
- `experiments/retrieval_experiments.md`: 上記実験の表形式サマリ。
- `validate_dataset_report.json`: データセット検証の詳細結果。
- `dataset_report.txt`: `analyze_dataset.py` のテキスト要約。
- `gnn_train_eval_report.json` / `gnn_refine_svd_report.json` / `baseline_eval_report.json`: 各スクリプトの評価サマリ (実行時に更新)。
- `input_pdfs/`: 一括抽出の入力 PDF を置くディレクトリ。
- `figures/`: 分析スクリプトが出力する図 (PNG 等)。

4.4 ドキュメント

- docs/development_log.tex：本ドキュメント（開発記録の原本）。
- docs/README.md：development_log.tex を PDF にビルドする手順。
- docs/GITHUB_SYNC.md：リモート同期に関する補足メモ。

5 データセット定義と評価（次の開発者向けガイド）

5.1 データ構造（Equation と Training Case）

Equation (unified_equations.json)

- 1 要素が 1 つの数式（ノード）で、主なキーは source_id, eq_id, equation, variables, context_text, domain。
- variables は辞書で、キーが変数シンボル（例: C_A）、値が物理意味の説明文（例: Concentration of component A）。
- **グローバル ID の注意**: eq_id は文書内ローカルのことがあるため、訓練や評価では **必ず** <source_id>__<eq_id> 形式（例: handbook_chemeng__eq_1）を「式 ID」として扱う。

Training Case (training_cases.json)

- 1 要素が 1 つの推論タスクで、主なキーは case_id, variant_type, context, input_variables, output_variables, correct_model_ids。
- correct_model_ids は「このケースの正解となる数式集合（1 本以上）」であり、各要素は **必ず** <source_id>__<eq_id> 形式。
- variant_type: original, swap_io, random_io_from_models, context_paraphrased (I/O と正解モデルを保持し文脈だけ言い換え)。

5.2 タスク定義（Equation Retrieval / Model Selection）

本研究の基本タスクは、与えられたケース (context, input_variables, output_variables) から、式集合ライブラリ（全式）内の correct_model_ids を上位にランキングする **retrieval** として定義する。各ケースは正解が 1 本のことも複数本のこともある（multi-positive）。

5.3 なぜ MRR を使うか（採用理由）

本タスクでは「正解が上位に出てくること」が重要であり、特に **1 本目の正解** が何位に現れるかがユーザー体験（候補式提示）に直結する。また、ケースによって正解式が複数あるため、分類精度（accuracy）のような単一ラベル前提の指標は不適切である。そのため、ランキングの早期精度を評価できる **MRR (Mean Reciprocal Rank)** と **Recall@K** を主要指標として採用した。

5.4 MRR と Recall@K の計算方法

各ケース i について、モデルが全式をスコアリングし降順に並べた順位列を考える。正解集合を G_i (correct_model_ids に対応するノード集合) とする。

- **BestRank**: $r_i = \min\{\text{rank}(e) \mid e \in G_i\}$ （正解集合のうち最も上位に来た式の順位）
- **Reciprocal Rank**: $RR_i = \frac{1}{r_i}$
- **MRR**: $\text{MRR} = \frac{1}{N} \sum_{i=1}^N RR_i$

- **Recall@K** : $\text{Recall@K} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[r_i \leq K]$

ここで N は評価対象ケース数。正解が複数ある場合でも、最初に当たる正解の順位 (BestRank) で評価する。

5.5 データ分割 (リーク防止のための paper/source_id split)

同一文書から抽出された式や文脈が train と test に混ざると、文字列一致により過大評価 (リーク) になる。そのため、分割は **source_id (論文/教科書) 単位**で行い、case の correct_model_ids から source_id を推定して train/val/test に割り当てる。実装例は run_retrieval_experiments.py (experiments/retrieval_experiments.json に split を保存)。

5.6 再現可能な実験手順 (何をどう動かすか)

- **健全性チェック**: python validate_dataset.py (式 ID 参照、I/O 変数整合、重複など。レポートは validate_dataset_report.json)。
- **訓練ケース増強 (context のみ言い換え)**: python add_context_paraphrases.py (training_cases.json を更新)。
- **統計と図の更新**: python analyze_dataset.py (dataset_report.txt と figures/)。
- **方式比較とハイパラ探索**: python run_retrieval_experiments.py (結果は experiments/retrieval_experiments.json)。

5.7 現時点の結論 (次に何をすべきか)

現状では、学習を伴う GNN/MLP 系よりも、**TF-IDF (文脈) + 変数集合 Jaccard (I/O 一致)** を混合した baseline_mix が最良である。次の開発では、(1) negative sampling を hard negative 化、(2) 変数をノードとして明示した二部グラフ化、(3) 「正解式集合の整合性 (最小十分性)」を利用した学習目的の改善、などによりベースライン超えを狙う。

6 GNN の仕組みと埋め込み (次の開発者向け)

6.1 なぜ現状のグラフ構築では GNN の強みを活かせないか (分析)

GNN の強みは、タスクに効く構造をグラフに載せ、その上で局所的な集約をしてノード表現を磨くことにある。当初の「式をノード、共通変数があればエッジ」という設計には、次のような限界がある。

(1) エッジが極端に密になり、構造の弁別力が弱い

- ノード数 N に対し、変数を 1 つでも共有する式ペアに無向エッジを張ると、 t, T, V, C_A など頻出変数を含む式が多数つながる。実際のデータでは 975 ノードで約 16 万本のエッジとなり、1 式あたり平均で非常に多くの式と隣接する。
- その結果、「同じ物理モデルに属する式」と「たまたま t や T を共有するだけの式」が区別されず、メッセージパッシングで無関係な情報が混ざり、ノード表現がシャープに磨かれない。

(2) 変数共有と retrieval の正解が対応していない

- タスクは「与えられたケース (context + I/O 変数) に合う式をランキングする」retrieval である。正解は「そのケースの入出力を満たす式集合」であり、変数名の共有は「候補になりうる」必要条件にすぎない。
- 多くの不正解の式も t や T を共有するため、グラフのつながりだけでは「このクエリに正解」というシグナルが弱く、GNN の集約が retrieval に直結しにくい。

(3) 変数がグラフ上に明示されていない

- 変数はエッジの「条件」としてしか現れず、ノードとしては存在しない。そのため「この式はクエリの

出力変数 C_A を含む」といった情報を、メッセージパッシングで明示的に扱えない。のちにクエリ条件 (I/O 変数) に依存した重み付けやサブグラフを入れることも難しい。

(4) ホモフィリー仮定が成り立ちにくい

- 多くの GCN 系モデルは「つながっているノードは似ている」という仮定で学習する。ここでは「つながり = 1 つでも変数名が被った」なので、ドメインが違っても t だけ共有でつながる、といったノイズが多く、ホモフィリーが弱い。そのため、隣接ノードからの集約が「関連の強い式」に絞られず、GNN の利点が生かしづらい。

以上から、「式 = ノード・共通変数 = エッジ」だけの同質グラフは、retrieval というタスクに対して GNN の強みを十分に引き出せる設計にはなっていない、という結論になる。改善の方向性として、(i) 変数を明示した式-変数二部グラフで「どの変数を介してつながるか」を構造に載せる、(ii) クエリの I/O 変数に応じたエッジ重みやサブグラフにする、(iii) 訓練ラベルを利用して「同じ正解に共起する式」同士をエッジで結ぶ、などが考えられる。

6.2 グラフの構築 (build_graph_data.py → equation_graph.pt) 【二部グラフ版】

タスクに効く構造と局所集約を両立させるため、式ノードと変数ノードの二部グラフを採用する。

- ノード：式ノード (0 から $N_{eq} - 1$) と 変数ノード (N_{eq} から $N_{eq} + N_{var} - 1$)。式ノードは unified_equations.json の並びと一致。変数ノードは全式から出現する変数名のユニーク集合に 1 対 1 で対応する。
- エッジ：式 i が変数 v を含むときのみ、無向エッジ $(i, N_{eq} + j)$ と $(N_{eq} + j, i)$ を張る (j は変数 v の ID)。式同士は直接つながらず、変数ノードを介した 2 ホップで「同じ変数を含む式」に情報が伝わる。これにより集約が「どの変数を共有しているか」に沿った形になり、局所性も保たれやすい。
- ノード特徴 x ：式ノードには従来どおり CodeBERT の [CLS] ベクトル (768 次元)。変数ノードには、その変数を含む式の CodeBERT ベクトルの平均を 768 次元特徴として与える (学習可能な変数埋め込みにすることも可能)。 x の形状は $(N_{eq} + N_{var}, 768)$ 。
- 保存内容：Data($x=x$, edge_index=edge_index, num_equations= N_{eq}) に加え、後方互換のため式どうしの隣接 edge_index_eq_eq ($0..N_{eq} - 1$ のみのインデックス) も保存する。学習・評価では式埋め込みは GCN 出力の 先頭 N_{eq} 行のみを用いる。

6.3 式の埋め込みの 2 系統

Retrieval 実験では、式の表現に次の 2 系統がある (スクリプトによってどちらを使うかが異なる)。

系統 A：CodeBERT ノード特徴 (equation_graph.pt の x)

- 上記のとおり、事前学習済み CodeBERT で式 + 文脈をエンコードした 768 次元ベクトル。学習時には L2 正規化してコサイン類似度 = 内積として使う。
- train_gnn_retriever.py の「式側は x をそのまま (または GCN で更新)」や、run_retrieval_experiments.py の q_mlp_to_x で使用。クエリ側は SVD 空間のベクトルを MLP で 768 次元に写す方式。

系統 B：TF-IDF + TruncatedSVD (同一空間でクエリと式を比較)

- 式テキスト：context_text, equation, domain を連結した文字列を TF-IDF (TfidfVectorizer, lowercase, max_features=50000, ngram_range=(1,2)) でベクトル化し、TruncatedSVD で 256 次元に圧縮。L2 正規化したものを式埋め込み E_0 とする。
- クエリ (ケース) テキスト：context と INPUT <input_variables>, OUTPUT <output_variables> を連結し、同じ TF-IDF と SVD で 256 次元に写し L2 正規化したものをクエリ埋め込み Q とする。
- スコアは $E_0^T Q$ (内積 = コサイン類似度)。svd_cos ベースラインや train_gnn_refine_svd.py の入力はこの空間。

6.4 GNN の役割 (2 種類の使い方)

パターン 1: 式特徴を CodeBERT x のまま、GCN で更新 (train_gnn_retriever.py)

- 式埋め込みは $z = \text{GCN}(x, \text{edge_index})$ 。2 層 GCN (GCNConv(in_dim, 768), ReLU, Dropout, GCNConv(768, 768)) の出力を L2 正規化。クエリは SVD ベクトルを QueryEncoder (MLP) で 768 次元に写し、 $z^\top q$ でスコア。InfoNCE 風の負例サンプリングで学習。
- 現状の実験では、クエリと式が別空間 (SVD vs CodeBERT) から来るため学習が難しく、このパターン単体ではベースラインを下回っている。

パターン 2: SVD 空間で式埋め込みを GCN で残差更新 (train_gnn_refine_svd.py, gnn_refine_svd_residual)

- 式の初期埋め込みは系統 B の E_0 (256 次元、L2 正規化済み)。これを 2 層 GCN の入力とし、 $h = \text{GCN}(E_0)$ を計算したあと、**残差混合** $E = E_0 + \alpha \cdot h$ で更新 (α は学習可能スカラー、初期値 0 でクリップ)。最終的に E を L2 正規化して式埋め込みとする。クエリは同じ SVD 空間の Q のまま、スコアは $E^\top Q$ 。
- $\alpha = 0$ のときは GCN を通さないで SVD コサインと同一。グラフ構造 (変数共有) の情報を少しずつ取り込む設計。学習はサンプルされた負例に対する InfoNCE 的損失。

6.5 スコアリングと学習

- いずれの方式でも、式埋め込みとクエリ埋め込みは L2 正規化してから内積をとるため、スコアは **コサイン類似度** に等しい。
- 学習時: 各訓練ケースについて正解式を 1 本選び、ランダムに多数の負例式をサンプル。正解を 1 位にしたいので、正解・負例のスコアに温度 τ (例 0.07) をかけた logits に対して cross-entropy (正解インデックス 0) を最小化 (InfoNCE 型)。

6.6 ファイルと役割の対応

- build_graph_data.py: 式-変数二部グラフを構築。 x は式ノード+変数ノードの特徴、edge_index は二部エッジ、num_equations と edge_index_eq_eq (式同士の隣接) も保存。
- train_gnn_retriever.py: 式埋め込みは GCN/GAT 出力の先頭 num_equations 行を使用 (二部グラフ対応)。クエリ = SVD $\rightarrow$ MLP。-use-gnn / -no-gnn で GNN の有無、-gnn-type gcn|gat で GNN 種類を切替。
- train_gnn_refine_svd.py: 式 = SVD(E_0) を残差 GCN で更新、クエリ = SVD(Q)。edge_index_eq_eq (式同士のみの) を使用。
- run_retrieval_experiments.py: 上記に加え baseline_mix、svd_cos、q_mlp_to_x、gnn_refine_svd_residual を同一分割で比較。二部グラフ時は式ノードのみ $x[:\text{num_equations}]$ と edge_index_eq_eq を利用。

7 今後の予定・TODO

- 複数 PDF の一括処理と正規化の一括適用。
- 正規化ルール (notation_map.json) のドメイン別拡張。
- 抽出結果の検証用テスト (期待する数式との差分チェック)。
- GitHub との同期と CI (テスト・リント) の検討。

7.1 試したいこと・実験アイデア（大胆に）

- **GNN の種類**：GCN に加え GAT（Graph Attention）を試す。二部グラフでは「どの変数ノードを重視するか」を attention で学習できると有利な可能性。
- **エッジ重み**：変数ごとの IDF や、クエリ I/O 変数との一致度でエッジに重みを付け、メッセージの強さを変える。
- **クエリ条件付きグラフ**：推論時に「このクエリの入力・出力変数を含む式／変数」にだけメッセージを流す、あるいはエッジをマスクする。タスクに効く構造に絞る。
- **Hard negative**：ランダム負例に加え、変数は被るが正解でない式を意図的に負例にし、境界を学習しやすくする。
- **式の言い換え・拡張**：同一式の context 言い換えや変数名の正規化違いをデータ拡張し、埋め込みのロバスト性を高める。
- **評価の安定化**：同じ分割で複数 seed を回し、平均・標準偏差で報告。小規模データなのでばらつきを明示する。

8 更新履歴

- 2026-06-02（進捗報告 v6 作成：データ拡大・難易度勾配・仮説 B 再検定・頑健性・LLM 同一条件比較の統合）：前回 v5（3,244 式・1,107 ケース）以降の Phase 3-5 成果と外部比較を**差分版の進捗報告 v6**としてまとめ、docs/progress_report_2026-06-02_v6.tex (lualatex / ltjsarticle, 8 ページ)を新規作成。主な内容：(1) データ規模 11,146 式・2,838 ケース (DAE 1,000 件含む), (2) 難易度勾配で本手法の Recall@K_correct 優位が標準 +0.127 → +DAE +0.210 → DAE のみ +0.217 と拡大, (3) 仮説 B 再検定でフル DB が Recall@K_correct で有意 (+0.048, $p = 0.044$, Cohen's $d = 0.74$ 中; MRR_first では飽和し差なし)と判明し v5 の暫定結論「DB 規模の影響なし」を訂正, (4) ハイパラ sweep 13 設定で Recall@K_correct レンジ 0.0094 (シード間 std の約 1/10)と頑健性を確認, (5) **外部ベースライン (LLM 直接生成) との同一条件比較を現行 11,146 式 DB で整合. LLM 比較の再評価手順**：LLM 評価済みの単一ソース 29 件・複数ソース 20 件サブセットに対し、`set_aware_reranker.py --modes baseline,reranker-10S --seeds 10 --save-per-case` で自然 5 variant (original / context_paraphrased / random_io_from_models / multisource_original / multisource_random_io) 学習・per-case 出力を生成し (experiments/phase6_full_perCase.json), 各対象ケースを**テスト分割に入ったシードのみ**で集計 (リーク防止の held-out 評価) する `aggregate_llm_subset_v6.py` を新規作成 (出力 experiments/phase6_llm_subset_comparison.json)。結果 (Recall@K_correct)：単一ソースは baseline 0.982 / reranker-10S 0.975 がいずれも LLM 0.707 を圧倒, 複数ソースは baseline 0.575 < LLM 0.842 < **reranker-10S** 0.942 となり, 古典 IR 単独では複数文献の組合せに不足し Set-aware リランカーが本質的に必要であることを同一条件で実証。なお和文太字は明朝に太字ウェイトが無いため BoldFont を角ゴシックに設定した。**（同日追記）** v5 の構成を参考に v6 を図駆動で簡潔化 (8 → 5 ページ; 目次・重複表を図に統合) し, 機械学習研究者視点の図 4 点を新規作成 (generate_figures_v6.py → docs/figures/fig_v6_{dataset,gradient,impact,llm}.png)：(A) データセット内訳・難易度分布, (B) 難易度勾配 [中核], (C) 正解式数別の改善+指標飽和 (仮説 B) の対比, (D) LLM 三者比較。各図はシード分散のエラーバー・差分注記・統一配色 (baseline 灰/本手法 青/LLM 橙) で「1 図 1 主張」とした。**（同日追記 2：LLM 難ケース比較・精度向上・アブレーション）** 加藤先生の「LLM で解けるなら不要では」「図 B に LLM も並べよ」とのご

指摘を受け, `evaluate_llm_dae.py` を新規作成し DAE 難ケースで LLM 直接生成 (Claude) を評価. `generate_figures_v6.py` の図 B を **3 者比較** (baseline / LLM / reranker-10S) に更新し, **全難易度**で **reranker > baseline > LLM** (最難 DAE で LLM 0.264 に対し本手法 0.699, 約 2.6 倍) を示した. 精度向上として Stage1 の retrieval 天井を診断 (DAE Recall@50 = 0.86, 高 X で取りこぼし X=10 で 0.68) し, 候補数 top-k を 50 → 200 に拡大して reranker-10S を再評価, **実用 Recall@20** を 0.912 → 0.968 に改善した (`topk_dae_{50,100,200}.json`; 厳格 Recall@K_correct は +0.02 で, Stage2 の順位付けが律速と判明). Set-aware 3 特徴量のアブレーション (`ablation_setaware.json`) で **gComp・gCoh が主力, gDom は寄与薄**と判明. 図 E (シード分散)・F (アブレーション)・G (精度向上) を追加し v6 は 7 ページに. 画像内タイトルの図番号は LaTeX キャプション採番へ一本化した.

- 2026-06-01 (Phase 5 完了: reranker ハイパーパラメータ sweep と頑健性の確認): `set_aware_reranker.py` に `--hidden-dim`, `--margin`, `--batch-size`, `--n-neg-samples`, `--weight-decay` を追加し, sweep 実行用 `run_phase5_sweep.py` (config を子プロセスで並列実行・結果集約) と分析用 `analyze_phase5_sweep.py` (ランキング・パラメータ感度・best 抽出, 複数 sweep を `--config-glob` で横断結合可) を新規作成. GPU サーバ初期化補助 `setup_gpu_server.sh` (rsync + Docker コンテナ + 依存導入 + CUDA 確認 + sweep キックオフ) も作成. **実行構成**: 固定軸 reranker-10S + 10 seed + 訓練データ (original + multisource_v3 + dae_X*), 探索軸 $lr \in \{1e-4, 3e-4, 1e-3, 3e-3, 1e-2\}$, $hidden_dim \in \{64, 128, 256\}$, $epochs \in \{20, 40\}$, $margin \in \{0.1, 0.3\}$. 計 13 config を評価 (A100×4 サーバ 8 config + ローカル lr sweep 5 config). **結果 (主指標 Recall@K_correct)**: 全 13 config が 0.7321–0.7415 に収まり, **config 間レンジ 0.0094 は seed 間標準偏差 0.1008 の約 1/10**. すなわち探索したいずれのハイパーパラメータ (lr を 100 倍振っても, $hidden_dim \cdot epochs \cdot margin$ を変えても) **統計的に有意な差を生まない**. best config は $lr=3e-4$, $hidden_dim=64$, $epochs=40$, $margin=0.1$ ($R@K_c = 0.7415$) だが既定値 ($lr=1e-3$, $hidden=64$, $epochs=15$) との差は誤差範囲. **結論**: reranker-10S の強さはハイパーパラメータの微調整に依存せず**頑健**であり, 「脆弱なチューニング由来の見かけ上の性能」ではないことを定量的に確認した (査読対策として重要). **運用上の知見**: 本手法は MLP が小さく **GPU 加速の恩恵がほぼ無い CPU-bound** 処理であり, 共有 GPU サーバ (過負荷時 load 53/16 コア) より**非競合のローカル実行の方が予測可能**. 長時間 CPU ジョブは `caffeinate -i` でスリープ抑止が必須. 保存先: `experiments/phase5_sweep/` (config 個別 JSON), `experiments/phase5_best.json`.
- 2026-05-29 (Phase 4 完了: 特性別性能分析 + 仮説 B 再検定): (1) **仮説 B (Full DB vs Narrow DB の本手法 MRR 差) を 3 指標で再検定**. 旧結果は `MRR_first` のみで $p = 0.78$ (n.s.) と「DB 規模の影響なし」と結論したが, `MRR_first` は天井 1.0 付近で飽和する指標であった. 新指標 `Recall@K_correct` (K = 正解式数 $n_{correct}$, R-Precision と同義) で同検定を行うと, **Full DB 0.9229 vs Narrow DB 0.8749**, 差 +0.0480 (paired t: $t = 2.34$, $p = 0.044$; Wilcoxon $p = 0.037$; Cohen's $d = 0.74$ medium) と**中程度効果の有意差**を確認. **MAP** も $0.953 \rightarrow 0.931$ ($p = 0.096$, $d = 0.59$ medium) と境界域で同方向の差. これにより「サチった指標が差を隠していた」ことが明確になり, DB 拡張の効果は実際にはあるが評価指標の選択によって見え方が変わる, という重要な知見を得た. 保存先: `experiments/phase4_hypothesis_b_test.json`. (2) **特性別 (stratified) 性能分析**. `set_aware_reranker.py` に `--save-per-case` フラグを追加して各ケースの評価結果を JSON にダンプする機能を実装し, 10 seed・1,592 ケース・1,223 ユニーク (original + multisource_v3 + dae_X1..X10) について baseline vs reranker-10S を 4 軸 (正解式数 $n_{correct}$, 横断ソース数 $n_{sources}$, 入力変数数 n_{input} , 出力変数数 n_{output}) で層別化. 主要結果 (差は `Recall@K_correct`): $n_{correct} = 1$ では +0.02 と差なしだが, $n_{correct} = 2$ で +0.21, = 3 で +0.26, 5-10 で +0.24 と**複数次ケースで大幅改善**. $n_{input} = 1-3$ では -0.02 (baseline がすでに

- 0.92) で本手法不要, 11+ で +0.25 (baseline 0.45 $\rightarrow$ reranker 0.70). **Recall@K_correct** は MRR_first の天井問題を解消し, 提案手法の真の改善幅 (baseline 0.526 $\rightarrow$ reranker 0.735, +0.21) を露わにした. `analyze_phase4_characteristic.py` と `analyze_phase4.py` を新規作成・更新. 保存先: `experiments/phase4_characteristic_analysis.json`, 図 `docs/figures/fig_phase4_stratified.png`.
- 2026-05-28 (Phase 3 完了: 制約付き実験 A/B + DAE データセット): **(1) DAE 訓練ケース 1,000 件追加.** `generate_dae_cases.py` を新規作成し, Anthropic API (Claude Sonnet 4.5) で $X \in \{1, 2, \dots, 10\}$ の各値について 100 件ずつ生成 (合計 1,000 件, `variant_type = dae_X{N}`). アルゴリズム: (i) ランダムに ODE を 1 本選択し正解集合の初期種とする, (ii) その微分変数を出力初期値に追加, (iii) 同一ソース内の変数共有式を優先的に正解集合に追加, (iv) $|\text{正解集合}| = X$ になるまで反復, (v) $X > 1$ の場合は追加された各式の代表変数を出力に加えて自由度 0 を保証, (vi) 残り変数を入力とし context を Claude API で生成. 最終訓練ケース数: 1,838 $\rightarrow$ 2,838. **(2) Phase 3 制約付き実験.** `set_aware_reranker.py` に `--variants` フィルタ (プレフィックスマッチ対応) と `--output` オプションを追加. 実験 A (original + multisource_v3 + dae_X*) と B (dae_X* のみ) を 10 seed で実行. 結果: A で baseline MRR_first 0.8024 $\rightarrow$ reranker 0.9232 (+0.121), B で baseline 0.7806 $\rightarrow$ reranker 0.9057 (+0.125). **DAE のみに絞ると ベースラインの MRR_first が 0.78 まで低下** (フルデータでは 0.917) し, 本手法の改善幅が拡大 (MAP +0.30). DAE 系ケースが従来 random_io 拡張より格段に **難しいデータ**であることが定量的に示された. **(3) Phase 1 評価指標拡張.** `evaluate_multi_eq.py` に `Recall@K_correct` ($K = n_{\text{correct}}$) を追加. 従来の MRR_first は飽和しやすく改善幅が見えにくい問題を解消. Phase 3 結果での `Recall@K_correct`: A で 0.525 $\rightarrow$ 0.735 (+0.210, **+40% 相対**), B で 0.481 $\rightarrow$ 0.699 (+0.218, **+45% 相対**). 関連スクリプト・データ: `generate_dae_cases.py`, `run_phase3_experiments.sh`, `experiments/phase3_A_original_multi_dae.json`, `experiments/phase3_B_dae_only.json`.
 - 2026-04-18 (自由度検証済 1,107 ケースでの主要結果の再評価): `two_stage_query_conditioned.py` を 5 seed で再実行. **reranker-10 の優位性が消失**し, reranker-7 が最良となる大きな結果の変化を観測 (baseline 0.924 $\rightarrow$ 0.937, reranker-7 0.937 $\rightarrow$ **0.959**, reranker-10 0.949 $\rightarrow$ 0.939). 全体として各手法の MRR が向上したことから, 自由度検証により学習・評価双方の品質が改善していることが示された. reranker-10 のグラフ特徴量 (#7-9) は初回データの不正ケース (物理的に解けないケース) で正解順位を押し上げる役割を果たしていた可能性があり, 現行の特徴量設計の見直しが必要. 進捗レポート v2 § 5 に新しい結果表・バリエーション別・初回データとの差分を追加, § 6 考察に再解釈を追記, § 7 今後の課題に「グラフ特徴量の再設計」を高優先度で記載. 実験結果は `experiments/query_conditioned_comparison.json` に保存.
 - 2026-04-19 (追補レポート作成: 自由度解析 + Set-aware 特徴量): 前回報告 (v2) 後の一連の研究成果を追補報告としてまとめ, `docs/progress_report_2026-04-19_addendum.tex` を新規作成. 主な内容: (1) 自由度解析による訓練データ品質改善 (1,107 件を全件可解に再構成), (2) 既存グラフ特徴量 f7-f9 の診断 (f9 は AUC 0.087 で逆信号など) と限界の明確化, (3) Set-aware 3 特徴量 (gComp=補完性, gCoh=一貫性, gDom=ドメイン一致) の設計と評価, (4) 複数次評価指標 (MRR_worst, Recall@K, MAP) の導入. 10 seed paired t-test の結果: MAP 0.936 $\rightarrow$ 0.955 ($p = 0.004$), 複数次 Recall@3 (旧 FullRecall@3) 0.628 $\rightarrow$ 0.693 ($p = 0.032$), 複数次 MRR_worst 0.376 $\rightarrow$ 0.397 ($p = 0.024$). reranker-10S (基本 7 + set-aware 3) が統計的に有意な改善を達成. 論文文化の方向性を「Set-aware graph features for multi-equation retrieval」に確定. 関連スクリプト: `filter_solvable_cases.py`, `expand_solvable_random_io.py`, `diagnose_graph_features.py`, `graph_feature_ablation.py`, `evaluate_multi_eq.py`, `set_aware_reranker.py`. 実験結果: `experiments/graph_feature_diagnosis.json`, `experiments/graph_ablation_results.json`,

- experiments/multi_eq_evaluation.json, experiments/set_aware_results.json. **2026-05-08 追記**: FullRecall@K (全正解上位 K 内割合) から標準的な Recall@K ($|R_q \cap \text{Top}_K|/|R_q|$ の平均) に指標を切替. evaluate_multi_eq.py に Recall@K 計算を追加し再実行.
- 2026-04-18 (自由度=0 を保証する random_io の追加生成): 自由度フィルタで 707 件に減少した訓練データを, **自由度=0 を構成上保証する新規 random_io** を追加生成することで 1,107 件に還元. expand_solvable_random_io.py を新規作成し, 各コアの正解式変数集合 V と方程式数 n_{eq} に対し, V から n_{eq} 個を出力として選び残りを入力とする全組合せから, 既存の入出力組と重複しないものを採用する方式で 400 件生成. 新規 400 件全てが自由度解析をパス. 最終内訳: original 93, paraphrased 279, swap_io 2, random_io 733 (元 333 + 新規 400) = 1,107 件. 進捗レポート v2 § 3.4.2 を「自由度解析+可解ケース追加生成」に再改訂, § 2.1 概要表・§ 4.5 正例・§ 5 結果注記・§ 7 今後の課題の数値を 1,107 に更新.
 - 2026-04-18 (自由度解析による訓練ケースのフィルタ実装): 加藤先生指摘「物理的に解けないケースは学習に使ってはいけない」への対応. filter_solvable_cases.py を新規作成し, 各訓練ケースについて正解式集合の変数数と独立方程式数を照合する自由度解析を実装. 判定条件: (i) 入出力変数が全て式集合内にあること, (ii) 出力変数が未知側にあること, (iii) $|\text{未知}| \leq \text{方程式数}$ (relaxed: 過剰決定系も物理的に解けるため許容). 訓練データ 1,107 ケースに適用した結果, 707 ケース (63.9%) のみが可解と判定されたため, training_cases.json を差し替え (元データは training_cases.pre_dof_filter.bak.json に退避). 内訳: original 123→93, context_paraphrased 369→279, swap_io 123→2 (98% 除外), random_io 492→333. swap_io は入出力非対称の元ケースが大多数であったため激減. 進捗レポート v2 § 3.4.2 を「自由度解析の実装済み」へ改訂, § 2.1 概要表・§ 4.5 正例定義・§ 5 結果の注記・§ 7 今後の課題に反映. MRR 再評価は今後の課題として明示.
 - 2026-04-17 (進捗レポート v2 の § 2.1 拡充: 変数表記揺れの方針明記): 前回加藤先生に「変数表記の正規化に取り組む」と述べた件について, 実データ (3,244 式) を精査した結果を受け, **現段階では全面正規化を行わない判断**に改めた旨を progress_report_2026-04-13_v2.tex の § 2.1 に明文化. 表記揺れの実態 (比熱が $c_p/C_p/c_w/C_p$ の 4 種 99 回、前指数因子が 3 種 31 回、密度が 3 種 251 回) を表で示し, 全面正規化を行わない 3 つの根拠 ((1) 記号の多義性による誤正規化リスク、(2) 文献単位の分割評価下では同文献内の表記が一貫し効果限定的、(3) 現 MRR 0.949 で誤差の主因でない可能性) を明記. 実装済みの normalize_notation.py (3 物理量ルール) によるアブレーション実験を今後の内部検証として位置づけ. レポート本文では具体的なファイル名への言及は削除し, 長期方針 (各変数の物理的意味の自然言語記述を埋め込み空間で比較し, 意味的に近いノード同士をグラフ上でソフトに接続する学習的アプローチ) のみを記載.
 - 2026-04-17 (進捗レポート v2 への注釈付与機構の刷新): v2 PDF へのハイライト+スティッキーノート付与を, 従来の「固定文字列マッチ」方式から**提出版 PDF とのテキスト単語レベル差分**方式に全面変更 (docs/annotate_report_v2.py を新規作成, 旧 add_annotations.py/add_annotations2.py を置換). pymupdf の get_text("words") で両 PDF の単語列を取り, difflib.SequenceMatcher で差分を取ることで, **実際に変更された箇所のみを正確にハイライト** (タイトルなど未変更箇所の誤ハイライトを解消). 正規化は Unicode NFC、ダッシュ類・引用符類・全角句読点・各種空白を統一し表記揺れを吸収. 結果: 321 箇所のハイライト (equal=1,234 単語・insert=217 単語・replace=242 単語) と 25 箇所のスティッキーノート (各修正に対応する加藤先生コメント内容を詳述).
 - 2026-04-09 (Phase 1 Round 2: 3,244 式への拡大 + 再評価): expand_dataset.py にハンドブック第 2 弾 (新 10 ドメイン+既存 6 ドメインの応用式、計 320 式) と大型 PDF 分割抽出 (pypdf で 80 ページずつ分割、3 冊 39 チャンク) を追加. 結果: **1,613 式 → 3,244 式**、ソース 52 → 71、変数 2,098 → 3,704. generate_training_cases.py を大規模カタログ対応に改修 (ドメイン別バッチ分

割 + 503 リトライ)。123 コア → 1,107 訓練ケースで 5 seed 比較：

- baseline: MRR 0.924 ± 0.040 , original 0.921
- reranker-7: MRR 0.937 ± 0.018 , original 0.942
- reranker-10 (graph): MRR **0.949 ± 0.031** , original **0.952**

核心的発見：(a) グラフ特徴量 (reranker-10) が全指標・全 variant で最良、(b) reranker-7 を明確に上回る (+1.2% overall, +1.0% original)、(c) 975 式 → 3,244 式のスケールでグラフ特徴量の有効性が単調増加。「データ規模がグラフ構造の価値を解放する」仮説を強く支持。Gemini API キーをハードコードから環境変数専用に変更済み。

- 2026-03-31 (Phase 1: データ拡大 + 再評価) : `expand_dataset.py` を新規作成し、3 つの手法でデータセットを拡大。(1) ハンドブック式的大幅拡張：LLM 知識から 15 新ドメイン (流体力学・伝熱・物質移動・熱力学・反応工学・プロセス制御・電気化学・バイオプロセス・環境工学・振動・電気工学・高分子・燃焼・分離・輸送現象) で 330 式を生成。(2) Semantic Scholar API で新論文を探索、open-access PDF 17 本をダウンロード。(3) 新規 + 既存未処理 PDF 15 本から 308 式を抽出。結果：**975 式 → 1,613 式 (+65%)**、ソース 22 → 52。訓練ケースも再生成 (154 → 187 コア、1,343 → 1,680 拡張)。拡大データでの再評価結果 (5 seed)：

- baseline: MRR 0.900 ± 0.069 , original 0.885 (旧: 0.875, 0.839)
- reranker-7: MRR 0.942 ± 0.052 , original 0.936 (旧: 0.885, 0.860)
- reranker-10 (graph): MRR **0.948 ± 0.046** , original 0.935 (旧: 0.879, 0.842)

核心的発見：データ拡大により (a) リランカーがベースラインを明確に上回る (+4-5%、旧 +1% 未満)、(b) グラフ特徴量が初めて悪化せず全体 MRR で最良、(c) 分散が半減 ($0.119 \rightarrow 0.052$)。「モデル改善より先にデータの質と量が重要」という仮説を実証。また、ハードコードされた Gemini API キーが GitHub 公開でリーク報告されたため、全 5 ファイルから削除し環境変数のみに変更。

- 2026-03-30 (Phase 3b: クエリ条件付きグラフ特徴量) : `two_stage_query_conditioned.py` を新規作成。二部グラフ (式-変数) の構造情報をクエリ条件付きで特徴化する 3 指標を設計：(1) 2-hop reachability (クエリ変数から候補式への到達率)、(2) graph neighbor overlap (候補式のグラフ近傍とクエリ変数関連式の Jaccard)、(3) indirect bridge ratio (候補変数がクエリ変数関連式にブリッジする比率)。7 次元リランカーに 3 特徴を追加した 10 次元リランカーで 5 seed 比較。結果：**クエリ条件付き特徴量も改善に寄与せず**。MRR 0.879 ± 0.123 (7 次元の 0.883 ± 0.119 から微低下)、original MRR 0.842 (7 次元の 0.860 から -0.018)。GCN ベース特徴量 (Phase 3) と同様のパターン。**結論：**現データ規模 (975 式・154 original ケース) ではグラフ構造が有効な追加信号を提供しない。7 次元学習リランカー (no-gnn) が最良手法として確定。論文では「なぜグラフが効かないか」の分析を contribution に含める。
- 2026-03-30 (Phase 3: GNN 特徴量統合) : `two_stage_retrieval_gnn.py` を新規作成し、3 方式を 5 seed で比較。(1) baseline_mix (固定重み)、(2) no-gnn リランカー (7 次元特徴、学習済み MLP)、(3) gnn リランカー (+GCN-refined SVD/CodeBERT 特徴の 9 次元)。結果：**no-gnn リランカーが最良** (MRR 0.885 ± 0.113 , original 0.857)。GNN 特徴を追加すると分散が増大し性能低下 (MRR 0.873 ± 0.148)。原因分析：(a) CodeBERT GCN のセルフシフト特徴はケース依存情報を持たない、(b) SVD GCN の学習が不十分 (5 epoch, 64 sample)、(c) グラフ構造がクエリ固有の情報を反映していない。**結論：**特徴の重み付けを固定 → 学習に変えるだけで改善が得られるが、現状のグラフ構造から有効な特徴を抽出するにはクエリ条件付き GNN が必要。
- 2026-03-30 (Phase 2: 2 段階 retrieval) : `two_stage_retrieval.py` を新規作成。Stage1 で baseline_mix が Top-K ($K=50$) 候補を高速に取得し、Stage2 でニューラルリランカー (7 次元特徴量、3 層 MLP、ペアワイズ margin loss) が精密にリランキング。5 seed 評価の結果、全体 MRR $0.875 \rightarrow 0.881 (+0.006)$ 。**original ケースでは** MRR $0.839 \rightarrow 0.856 (+0.017)$ と改善を確認。ただ

し seed 456 で劣化するなど分散が大きく、統計的有意性は未達。

- 2026-03-30 (データ品質修正): `fix_dataset_quality.py` を新規作成。暗黙変数 (s, t) を 277 式に追加、重複ケース 43 件を除去 (1386 $\rightarrow$ 1343)。バックアップを `*.bak.json` に保存。
- 2026-03-30 (Phase 0: 評価基盤の確立): 論文化を見据え、研究の土台を再設計。
 - **ベースライン失敗分析** (`analyze_baseline_failures.py` 新規作成): 全 1386 ケースで `baseline_mix` を評価し、失敗ケース ($RR < 0.5$) の 85 件 (6.1%) を 4 パターンに分類。**`cross_domain`** (100%) と **`context_divergence`** (78%) が支配的 — クエリと式の語彙が異なるケースでベースラインが失敗しており、意味理解 (GNN/ニューラル) が有効な領域を特定。可視化を `figures/failure_analysis_summary.png` に出力。
 - **複数 seed 評価の導入** (`run_retrieval_experiments.py` 改修): 5 seed (42, 123, 456, 789, 1024) で `source_id` 分割を変え、平均 $\pm$ 標準偏差と 95% bootstrap CI を報告する `-seeds N` モードを追加。結果: `baseline_mix` の Test MRR = 0.890 ± 0.095 (95% CI [0.804, 0.949])。単一 seed (0.899) では見えなかった高分散を確認。
 - **variant_type 別評価の標準化**: original ケースの MRR = 0.851 ± 0.080 に対し、`random_io_from_models` の MRR = 0.925 ± 0.112 と合成ケースが有意に簡単であることを定量的に確認。論文では original ケースの指標を主要報告とする方針。
- 2026-03-30 (TeX): `development_log.tex` の inputenc を UTF-8 のみに整理し、更新履歴の Markdown 風バッククォートをやめてファイル名をタイプライタ体コマンドで囲む表記に修正、`uplatex` でコンパイル可能に。TeX は `brew install texlive` で `pdflatex/uplatex/dvipdfmx` を利用 (手順は `docs/README.md`)。
- 2026-03-30: Claude Code 向け `CLAUDE.md` と `research_context.md` のポインタを追加。各ファイル・ディレクトリの役割を新設し各ファイルの役割を記載。技術メモの `build_graph_data.py` を二部グラフ版の説明に修正し当該節へ参照。
- 2026-03-18 (試したいこと・GAT 追加): 開発記録に「試したいこと・実験アイデア」を追加 (GAT、エッジ重み、クエリ条件付きグラフ、hard negative、データ拡張、複数 seed 評価)。`train_gnn_retriever.py` に GAT エンコーダと `-gnn-type gcn|gat` を追加。
- 2026-03-18 (グラフ構築の改善): 現状の「式=ノード・共通変数=エッジ」では GNN の強みを活かさないという分析を開発記録に全文追記。**式-変数二部グラフ**に変更 (`build_graph_data.py`)。式ノードと変数ノードを明示し、エッジは「式が変数を含む」のみ。式埋め込みは GCN 出力の先頭 `num_equations` 行を使用。`edge_index_eq_eq` で後方互換を確保。`train_gnn_retriever.py`、`run_retrieval_experiments.py`、`train_gnn_refine_svd.py`、`analyze_dataset.py` を二部グラフ対応に修正。
- 2026-03-18 (GNN・埋め込みの説明): 開発記録に「GNN の仕組みと埋め込み」セクションを追加。グラフ構築 (ノード=式・エッジ=変数共有・特徴=CodeBERT)、式埋め込みの 2 系統 (CodeBERT x と TF-IDF+SVD)、GNN の 2 パターン (x を GCN で更新する方式と SVD 空間での残差 GCN)、スコアリング・学習の流れ、各スクリプトの役割を記載。
- 2026-03-18 (方式比較とハイパラ探索): retrieval 方式 (`baseline_mix`, `svd_cos`, `q_mlp_to_x`, `gnn_refine_svd_residual`) を同一の train/val/test (`source_id` 単位) で比較し、学習率・重み減衰のグリッド探索を自動実行するランナー `run_retrieval_experiments.py` を追加。結果は `experiments/retrieval_experiments.md` に表で出力し、現状では `baseline_mix` (TF-IDF+ 変数 Jaccard) が最良であることを確認。
- 2026-03-18 (GNN 次ステップ): SVD+I/O 文字列ベースラインを維持したままグラフ構造の寄与を検証するため、SVD 空間上で式埋め込みを GCN で**残差的に**更新する実験スクリプト `train_gnn_refine_svd.py` を追加。素朴な GCN は埋め込みを崩して性能低下したため、学習開始時

はベースライン同等 ($\alpha = 0$) になる残差混合に変更して安定化。

- 2026-03-18: データセット解析と訓練準備を拡張。dataset_report.txt の変数 TOP15 に物理意味を併記し、ドメインを論文用に合併して図 (ドメイン分布・埋め込み) を可読化。訓練ケースに context_paraphrased を追加し、健全性チェック (ID 整合・変数整合・重複) とベースライン評価を実装。さらに PyG の equation_graph.pt を用いた retriever の 1epoch 動作確認スクリプトを追加 (context+I/O 変数をクエリ特徴に含めて改善)。
- 運用ルール変更: 全ての変動を開発ログに書き、変更のたびに git commit と git push で GitHub へ自動同期するよう Cursor ルールを強化 (.cursor/rules/development-log.mdc および本ドキュメント技術メモを更新)。
- 開発ログに変動履歴を追記: モデル (gemini-2.5-pro 等)、デフォルト API キー、処理済み PDF 管理、fetch_papers.py、build_graph_data.py、generate_training_cases.py (correct_model_ids の条件含む)、analyze_dataset.py。GitHub 同期はコミット・push で行う。
- build_equation_graph.py を追加 (PDF 一括、Handbook 生成、unified_equations.json 統合)。処理済み PDF を processed_pdfs.json で管理し未抽出のみ処理するよう変更。
- 開発記録の自動追記ルール (Cursor) とコミット時自動 push (post-commit フック) を導入。
- 初版: モデル変更、レートリミット対策、 c_w /正規化の記録を追加。